

Day 15 Learning (01/04/2026)

#Core Concepts and Definitions

ORM (Object-Relational Mapping)

- A technique to **map Java objects (classes) to relational database tables**.
- **Example:** A Java class *User* maps to a user table in a database.
- Enables developers to interact with databases using object-oriented programming instead of SQL queries.
- Simplifies operations like **insert, update, retrieve, and delete** records automatically.
- Popular ORM frameworks include **Hibernate**.

JPA (Java Persistence API)

- A **specification (set of rules)** for ORM in Java, not an implementation itself.
- Provides interfaces and annotations to achieve ORM but requires an **implementation provider** (persistence provider) such as Hibernate, EclipseLink, or OpenJPA.
- JPA abstracts the persistence layer, allowing easier interaction with relational databases.

Persistence Provider / ORM Tools

- Concrete implementations of JPA specification.
- **Examples:** Hibernate, EclipseLink, OpenJPA.
- These tools handle the actual database operations behind JPA interfaces.

Spring Data JPA

- Built **on top of JPA** to simplify usage and provide higher-level abstractions and utility methods.
- It is **not a JPA implementation** but facilitates easier interaction with JPA-based data persistence.

#Relational Databases vs. MongoDB (NoSQL)

Relational Databases (e.g., MySQL)

- Use **predefined schemas** with data stored in tables.

- JPA and ORM frameworks are designed primarily for these databases.

MongoDB

- A **NoSQL, schema-less database** using collections of flexible, document-based data models.
- Does **not use JPA** because it lacks tables and schemas typical of relational databases.
- Instead, MongoDB uses tools like **Spring Data MongoDB** to interact with the database.

#Spring Data MongoDB

- Acts as a **persistence provider for MongoDB**, similar in concept to JPA but designed for NoSQL.
- Provides necessary abstractions and implementations for working with MongoDB in Spring applications.
- Requires adding dependencies to integrate MongoDB with Spring.
- Enables developers to perform database operations easily without using JPA.

#Database Interaction Methods in Spring Data

Two primary ways to interact with databases using Spring Data (both JPA and MongoDB):

Method	Description	Complexity
Query Method DSL	Simple, convenient way to create queries based on method naming conventions.	Easy to use
Criteria API	Flexible API for building queries programmatically, suitable for advanced requirements.	More complex

#Key Insights

- **ORM simplifies database operations** by bridging Java objects and tables in relational databases.
 - **JPA is a specification**; real work is done by providers like Hibernate.
 - **Spring Data JPA enhances JPA usability** but depends on an underlying JPA implementation.
 - **MongoDB does not support JPA** because it is NoSQL and schema-less; instead, Spring Data MongoDB is used.
 - **Spring Data MongoDB provides similar abstraction benefits** for NoSQL as Spring Data JPA does for relational databases.
 - Developers can choose between **Query Methods DSL for simplicity** or **Criteria API for flexibility**.
 - **MongoDB is favored** for its flexibility, ease of learning, and wide adoption.
-

#MongoDB Basic Commands

Terminal Environment

- **Windows PowerShell** is commonly used to run these commands.

Core Commands and Database Operations

1. **mongosh**
 - Used for the verification of **Mongo Shell** and connecting to the instance.
2. **show dbs**
 - Lists out all the **databases** available in the system.
3. **use <name_of_db>**
 - Used for **creating** a new database or **switching** to an existing one.
4. **show collections**
 - Lists out all the **collections** inside the currently active database.

CRUD Operations (Create, Read, Delete)

5. **db.student.insertOne({<JSON Data>})**
 - Query to insert the first **document/record** into the **student** collection.
 - **Note:** If the collection does not exist, MongoDB will automatically create it and then insert the data.
6. **db.<collection_name>.find().pretty()**
 - Fetches all the documents inside a collection in a **readable (formatted)** way.

7. **db.<collection_name>.deleteOne({"name":"payal"})**
 - Deletes the specific document where the field **name** is "payal".
8. **db.<collection_name>.find({"name": "payal"})**
 - Fetches the document(s) based on the **parameter** provided (Filter).

Important Note

Unique Identifier: MongoDB automatically provides a **unique _id** to every document object created inside a collection.

Summary Table

Command	Purpose
mongosh	Opens the MongoDB Shell
show dbs	Displays all databases
use <db>	Switches to or creates a database
insertOne()	Adds a single document to a collection
find().pretty()	Shows all records in a clean format
deleteOne()	Removes a record based on a condition
find({filter})	Searches for a specific record

